VINUNIVERSITY

Deployment — Đưa Agent Lên Cloud

AICB-P1 · Ngày 12 · Từ localhost đến production URL

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

“Bạn demo cho sếp thấy agent chạy trên laptop. Sếp hỏi: khi nào 100 người dùng được? — và liệu nó có ngốn hết ngân sách không?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Từ localhost đến production
2. **Agent vs deploy truyền thống**
3. Docker & containerization (2026)
4. Thách thức riêng của agent
5. **Agent chạy ở đâu: server/client/on-device**
6. Cloud options + managed runtimes (Tier 0)
7. Hosting MCP servers
8. API gateway & security
9. Scaling + frontier-scale serving
10. CI/CD & eval gates
11. **Nâng cao: production-grade (tùy chọn)**
12. Checklist + phụ lục lệnh/code
13. Lab 12 + preview Day 13

Mục Tiêu Ngày 12

- Hiểu **gap giữa dev và production**: dependencies, config, secrets, networking
- Viết **Dockerfile** hiện đại (multi-stage + uv + slim/distroless) để đóng gói agent
- Nắm **3 thứ agent phá vỡ web infra thông thường**: long-running, stateful, cost
- So sánh **cloud options** theo trục quan trọng nhất với agent: **request timeout**
- Biết về **managed agent runtimes** mới (Bedrock AgentCore, Vertex Agent Engine)
- Thiết kế **API gateway** + cost protection và deploy agent có **public URL** hoạt động

Artifact pack cần nộp

Agent đã được containerize và deploy lên cloud, có health check endpoint, basic authentication, cost guard, và accessible qua public URL

- 1 Dockerfile (multi-stage, `uv`, $<$500MB) + docker-compose cho agent + dependencies
- 1 deployed instance trên Railway hoặc Render
- 1 health check endpoint (`/health`) + streaming endpoint (SSE)
- 1 public URL mà bất kỳ ai cũng có thể truy cập và dùng agent

Từ Localhost Đến Production

01

Agent chạy trên máy mình khác rất xa với agent chạy cho 100 người — gap đó không chỉ là “copy code lên server”

Recap: Agent Đã Hoàn Chỉnh Nhưng Chỉ Ở Local

11 ngày đã build

- LLM API + prompt engineering
- RAG pipeline grounded
- Multi-agent + MCP
- UX + trust layer
- Guardrails + safety

Nhưng đang chạy trên

- localhost:8000
- API keys trong .env file
- Chỉ 1 user (chính mình)
- Không health check
- Tắt laptop = agent chết

Lưu ý: “It works on my machine” là câu nói nổi tiếng nhất trong lịch sử software engineering. Day 12 giải quyết đúng vấn đề này.

Dev Environment ≠ Production Environment

Khía cạnh	Dev (localhost)	Production
Dependencies	“pip install” thủ công	Đóng gói cùng container
Config	.env file trên máy	Environment variables, secrets manager
Networking	localhost:8000	HTTPS, domain, load balancer
Users	1 (chính mình)	N users đồng thời
Failure	Restart thủ công	Auto-restart, health check

Nguyên tắc

Environment parity: dev/staging/prod **càng giống nhau càng ít bug** khi deploy.

Agent Không Phải Web App Bình Thường

Một CRUD app trả lời trong $<$1s. Agent thì khác về bản chất — và đó là nguồn gốc của mọi thách thức deploy hôm nay.

1. Long-running

Reasoning loop chạy 10–60s+ (có khi vài phút). **Phá vỡ** timeout 29–60s của gateway/proxy.

2. Stateful

Có conversation memory + tool history. **Mâu thuẫn** với quy tắc "stateless process" của 12-factor.

3. Costly

Mỗi call gửi lại cả history $\rightarrow$ cost tăng **siêu tuyến tính** (50–1000$\times$ token so với chat).

Lưu ý: Giữ 3 tính chất này trong đầu suốt cả bài. Mỗi section sau giải quyết một hệ quả của chúng.

12-Factor App — Áp Dụng Cho AI Agent

4 nguyên tắc quan trọng nhất

1. **Config in env:** không hardcode API keys
2. **Stateless processes:** agent không giữ state trên instance
3. **Port binding:** export service via port
4. **Dev/prod parity:** giữ gap nhỏ nhất

Deployment checklist

- ☐ Secrets management
- ☐ Health check endpoint
- ☐ Structured logging
- ☐ Monitoring endpoint
- ☐ Graceful shutdown

Lưu ý: Factor VI (stateless) nói "không dùng sticky session". Agent có memory → *externalize state* (mục *Thách Thức Riêng Của Agent*), không bỏ nguyên tắc.

02

Agent Deploy vs Deploy Phần Mềm Truyền Thống

Tin tốt: bạn ship agent bằng **đúng cỗ máy** đã có (CI/CD, container, load balancer). Tin cần nhớ: có 3 thứ bị *định nghĩa lại* — và đó là nơi agent khác biệt

Cùng Cỗ Máy Ship, Khác Cái Hộp Bên Trong

VINUNIVERSITY

Giữ nguyên (đừng phát minh lại)

- CI/CD pipeline, build artifact
- Immutable image, canary/rolling
- IaC (Terraform)
- Load balancer, health check
- 12-factor, stateless web tier

Bị định nghĩa lại (cái mới)

- **Test:** eval gate, không phải exact-match
- **Hoá đơn:** token runtime, không CPU-hour
- **Dependency:** model ngoài, rate-limit, deprecate
- **State:** hội thoại, không DB row
- **GPU economics:** không CPU/RAM

Câu thần chú

Cùng cái hộp; **cái bên trong** và **cách quyết định "đạt"** mới là phần khác.

Before / After — Monolith CRUD vs Agent

	App CRUD truyền thống	AI Agent
Kiểm thử	Unit test, assert chính xác	**Eval gate** (golden set, LLM-judge), gate theo điểm
Chi phí	CPU/RAM-hour, đoán trước được	**Token/request**, chỉ biết sau khi chạy
Latency	Mili-giây, đồng bộ	Giây–phút, **streaming** (SSE) + async
State	DB rows	Hội thoại / memory (checkpointer)
Dependency	Bạn kiểm soát & tự version	**Model bên ngoài**: rate-limit, bị deprecate
Scaling unit	CPU/RAM	GPU + VRAM

Lưu ý: Luận điểm: dùng **cùng machinery** (CI/CD, canary, immutable image, IaC, LB) — nhưng định nghĩa lại **the test, the bill, the dependency**.

Harness Mới Là Sản Phẩm — Sáu Lớp Bọc Quanh Lời Gọi Model

Lớp	Harness phải cung cấp	Trong coding agent 2026	Mục
Vòng lặp + tools	Loop, tool schema, retry, giới hạn lượt	Cùng một loop, ba mặt: CLI / SDK / hosted	§1
Cô lập	FS + network là **hai lớp bật-tắt độc lập**, ép ở tầng OS	Seatbelt (macOS), bubblewrap+seccomp (Linux)	§13
Chính sách quyền	allow / ask / deny, cưỡng chế **ngoài** model	opencode chỉ có lớp policy, **không** sandbox OS	§13
Cấu hình theo repo	File đi cùng git, thứ tự ưu tiên rõ ràng	`AGENTS.md` (gần nhất thắng) / `CLAUDE.md`	§12
Phiên (state)	Session ID, resume, lưu **ngoài** process	Transcript trên đĩa; hosted lưu server-side	§4
Cổng không tương tác	Exit code + JSON + chi phí mỗi lần chạy	`claude -p`; `opencode serve` + OpenAPI	§12

Điểm mấu chốt

Lời gọi model chỉ là **một dòng trong sáu**. Prompt **không phải** biên giới bảo mật: chỉ dẫn trong prompt/`CLAUDE.md` “không thay đổi những gì Claude Code *cho phép*”.

Model Là Dependency Bạn Không Kiểm Soát

Khác thư viện bạn pin trong `requirements.txt`: model sống trên server của người khác, có thể bị **khai tử** và có **trần thông lượng** do nhà cung cấp đặt.

Pin & deprecation là việc của deploy

- Pin **model ID** + tắt auto-upgrade (như deploy artifact)
- Provider báo trước $\geq$60 ngày rồi **request fail**
- Thực tế: **Assistants API gỡ 26/8/2026**; `gpt-4o/4-turbo/3.5` tắt **23/10/2026**

Rate limit = trần thông lượng

- RPM / TPM theo **tier** của nhà cung cấp
- VD GPT-4o: *rate-limit tier* 1 = 500 RPM $\rightarrow$ tier 5 = 10k
- Trần do **vendor** đặt, không phải autoscaler của bạn
- $\rightarrow$ retry/backoff + nhiều key (mục Scaling)

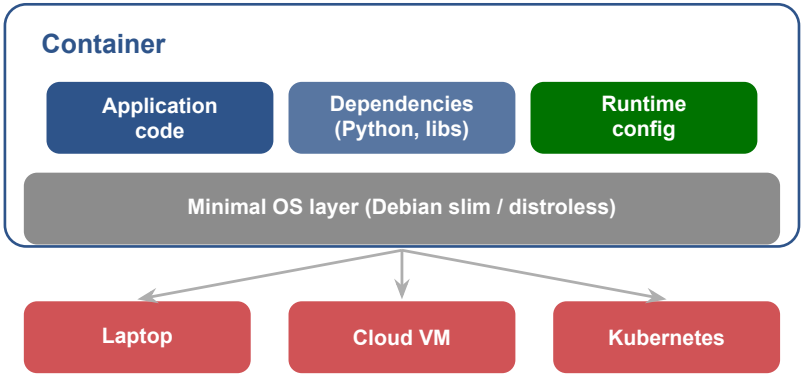
Lưu ý: Lịch **khai tử model** của nhà cung cấp là một forcing function lên release calendar của bạn.

Docker — Đóng Gói Agent Thành Container

03

Container giải quyết “it works on my machine” bằng cách đóng gói mọi thứ agent cần thành 1 unit chạy được ở bất kỳ đâu

Container Là Gì?



Ý chính: Container = app + deps + runtime đóng gói thành 1 unit. **Build 1 lần, chạy ở mọi nơi.**

Dockerfile 2026 — Multi-Stage + uv

```
# Stage 1: build deps with uv (Rust-fast, ~10x pip)
FROM python:3.12-slim AS builder
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
WORKDIR /app
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_PYTHON_DOWNLOADS=0
RUN --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
    uv sync --locked --no-install-project --no-editable
FROM python:3.12-slim          # Stage 2: slim runtime
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY . .
ENV PATH="/app/.venv/bin:$PATH"
RUN useradd -m app && chown -R app /app
USER app                       # non-root
CMD ["fastapi", "run", "main.py", "--host", "0.0.0.0"]
```

Lưu ý: Target <**500MB**: `uv`+cache, `--locked`, non-root, `.dockerignore`.

Base Image Showdown — Đừng Chọn Sai

Base image	Size (giải nén, trên đĩa)	Ghi chú cho AI agent
`python:3.12`	∼1.0 GB	Full — thừa toolchain, attack surface lớn
`python:3.12-slim`	∼150 MB	**Default tốt** cho ML Python (glibc, manylinux wheels)
distroless	∼66 MB	Gọn nhất + an toàn; không shell (debug khó)
`python:3.12-alpine`	∼55 MB	**TRÁNH cho ML** — xem cảnh báo

Lưu ý đơn vị: size "nén khi pull" nhỏ hơn ∼3–4× con số trên đĩa (vd `python:3.14-slim-trixie` ≈41 MB khi tải). Đừng so hai đơn vị với nhau.

Lưu ý: Alpine dùng `musl` libc → package **không có wheel `musllinux`** phải compile từ source (benchmark cũ pandas+matplotlib: slim 30s vs Alpine 26 phút). **2026 đã đỡ hơn**: numpy 2.5.1 / pandas 3.0.5 / matplotlib 3.11.1 đều đã có wheel `musllinux` cho CPython 3.12–3.14 — **nhưng matplotlib chưa có aarch64**, nên Alpine trên Apple Silicon/Graviton *vẫn* build từ source. Vẫn nên chọn slim.

Image Security — Scan, Non-Root, SBOM

3 việc bắt buộc

1. **Scan CVE:** Trivy / Docker Scout / Grype trước khi deploy
2. **Non-root:** `USER app`, không chạy root trong container
3. **Pin digest:** `FROM ...@sha256:...` thay vì tag mềm

SBOM — giấy tờ thành phần

SBOM (SPDX/CycloneDX): liệt kê mọi package trong image.

Sinh bằng `syft` / `docker buildx --sbom`.

Giờ là **yêu cầu pháp lý** (US EO 14028, EU CRA).

Mốc gần nhất: 11/9/2026 — CRA buộc báo ENISA **trong 24h**. Nghĩa vụ đầy đủ + SBOM: **11/12/2027**.

Lưu ý: Distroless ít CVE hơn nhưng **không phải zero**. Scan là việc lặp lại, không phải một lần.

Docker Compose — Multi-Service Setup

VINUNIVERSITY

Agent stack điển hình

- **Agent service:** FastAPI + LLM logic
- **Vector store:** Qdrant (6333/6334)
- **Cache:** Redis (6379) cho session/rate limit
- **Reverse proxy:** Nginx (optional)

Compose 2026

- `docker compose up` (V2, không gạch nối)
- Bỏ key `version:` (đã obsolete)
- `depends_on: condition: service_healthy`
- Service gọi nhau bằng tên (DNS): `qdrant:6333`

Cho lab

Bắt đầu với 2 services: **agent + vector store**. Thêm Redis và Nginx khi hệ thống cần scale.

Hình Hài Một Agent Service Tối Thiểu

```python
from fastapi import FastAPI
from sse_starlette.sse import EventSourceResponse

app = FastAPI()

@app.get("/healthz")        # health check for LB / Cloud Run
def healthz():
    return {"status": "ok"}

@app.post("/chat")          # streaming is the default, not the exception
async def chat(req: ChatRequest):
    async def gen():
        async for chunk in run_agent(req.messages):
            yield {"event": "token", "data": chunk}
        yield {"event": "done", "data": "[DONE]"}
    return EventSourceResponse(gen())
```

Lưu ý: `/healthz` cho hạ tầng biết agent sống; `/chat` **stream** token qua SSE.

Thách Thức Riêng Của Agent

04

Long-running + stateful = web infra thông thường “gãy”. Đây là phần mà một bài deploy bình thường bỏ qua — nhưng agent thì không thể

Vấn Đề Timeout — Agent Chạy Lâu Hơn Gateway Cho Phép

Reasoning loop của agent thường lâu hơn timeout mặc định của hạ tầng. Request bị cắt giữa chừng → user thấy lỗi 504.

Hạ tầng	Timeout mặc định	Ghi chú
AWS API Gateway	29s → 504	Có thể nâng (từ 6/2024)
Heroku router	30s (initial byte)	Không chỉnh được
AWS ALB (idle)	60s	Cắt stream im lặng
nginx `proxy_read_timeout`	60s	Giữa 2 lần đọc
Fly.io proxy (idle)	60s	Streaming reset timer
Railway public HTTP	15 phút	Private network: vô hạn

Lưu ý: 2 cách vượt: (1) **stream** từng token; (2) route qua **private network** hoặc **async job**. *Bảng đầy đủ theo platform: mục Cloud Deployment Options.*

Streaming Bằng SSE — Chuẩn De-Facto Cho LLM

Tại sao SSE, không phải WebSocket?

- Token streaming **một chiều** (server→client), chạy trên HTTP/1.1 thường
- `EventSource` tự reconnect + `Last-Event-ID`
- OpenAI & Anthropic đều dùng SSE (`stream:true`)

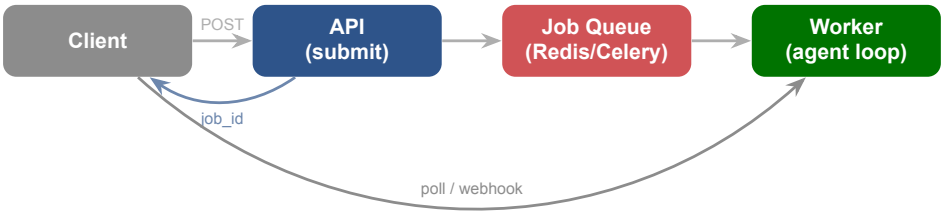
Lưu ý: 2 cái bẫy proxy hay gặp:

- nginx `proxy_buffering on` (mặc định) gom token lại → tắt nó, hoặc header `X-Accel-Buffering: no`
- Agent im lặng >60s → **heartbeat :** `ping` mỗi ∼15s để không bị cắt idle

Stream đứt giữa chừng?

Buffer token vào Redis theo `streamId` → reload thì **replay từ chỗ đứt**. **Giới hạn:** chỉ cứu reload trang.

Khi Quá Lâu — Chuyển Sang Async Job



- **Submit-and-poll:** API trả `job_id` ngay, client hỏi kết quả sau (hoặc nhận webhook)
- Tool: **Celery** (broker), **RQ** (Redis), **Cloud Tasks** (managed)
- Batch lớn không gấp? **Batch API** rẻ hơn **50%**: OpenAI trả trong **24h**; Anthropic cũng 50% nhưng phần lớn xong **dưới 1 giờ**

Statefulness — Mâu Thuẫn Với “Stateless”

12-factor nói agent phải stateless để scale. Nhưng agent *có* memory. Giải pháp: **externalize state**, không giữ trên instance.

Externalize ở đâu

- Conversation/session → **Redis / Postgres**
- LangGraph **checkpointer** (`PostgresSaver`) keyed bằng `thread_id`
- Bất kỳ instance nào cũng phục vụ được request → scale tự do

Durable execution (2025–26)

Cho agent chạy nhiều bước/nhiều ngày, resume sau crash:

- **Temporal** (replay) · **DBOS** (MIT, Postgres)
- **Inngest** (step memoization)
- **LangSmith Deployment** (tên cũ *LangGraph Platform*) — managed persistence

Lưu ý: “Sticky session” chỉ là **best-effort** — Cloud Run phá vỡ nó khi instance bị kill. Externalize state mới chắc.

Concurrency & Cold Start

Concurrency: dùng async

Mỗi request giữ worker **rất lâu** → dễ cạn worker pool.

- `async def`: 1 worker phục vụ nhiều request khi `await` I/O
- **Bẫy:** blocking call trong `async def` làm **nghẽn cả event loop**
- Cloud Run concurrency: mặc định 80, max 1000/instance

Cold start: ML deps nặng

Load model/embeddings lúc khởi động → cold start chậm.

- **Min instances** (Cloud Run) / **provisioned concurrency** (Lambda) giữ instance ấm
- **Lazy-load** object hiếm dùng ra khỏi cold path

Nguyên tắc agent

Agent **I/O-bound** (đợi LLM) → tăng concurrency tiết kiệm instance; giữ blocking code trong `def` thường.

Agent Chạy Ở Đâu? Server / Client / On-Device

05

Một trục kiến trúc deploy mà nhiều người bỏ qua: vòng lặp agent + tools + API key thực sự **chạy ở đâu**? Câu trả lời quyết định bảo mật, chi phí, và quyền riêng tư

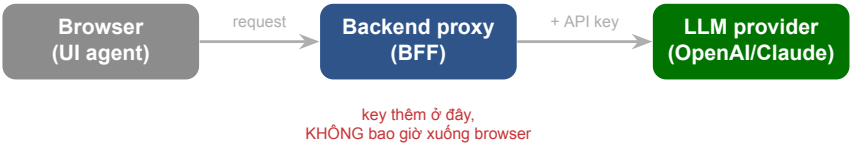
Phổ Vị Trí — Loop Chạy Ở Đâu

Vị trí	Loop chạy ở	API key	Cost/token	Năng lực
Server-side	Backend của bạn	An toàn (server)	Bạn trả hết	Frontier
Client (chỉ UI)	Trình duyệt, call **qua proxy**	Vẫn cần proxy	Bạn trả hết	Frontier
Local-first (BYOK)	Máy user (CLI/IDE)	Key **của user**	**User trả**	Frontier
In-browser model	Trình duyệt (WebGPU)	**Không cần**	**Bằng 0**	∼1–3B
On-device	NPU/GPU thiết bị	**Không cần**	**Bằng 0**	∼3B
Edge	Node biên (Workers AI)	Ở provider	Pay-per-call	∼7B+
Hybrid	On-device → escalate cloud	Cloud tin cậy	Rẻ→đắt	Nhỏ→frontier

Mặc định

Đa số agent production là **server-side**: loop + tools + key ở backend, client chỉ là lớp mỏng gửi message. Đổi lại: mọi token đi vòng qua server bạn, bạn trả toàn bộ compute.

Điểm Mấu Chốt: “Client-Side” Hiếm Khi Là Keyless



Lưu ý: Build tool **nhúng** biến `VITE_`/`NEXT_PUBLIC_` thẳng vào JS bundle → key ship xuống trình duyệt và **bị lấy cắp**. Vì vậy ngay cả agent “client-side” với model frontier **vẫn cần backend proxy** (Backend-for-Frontend) + rate limit. Chỉ **on-device / in-browser model** mới thật sự keyless.

On-Device & Edge — Keyless Thật Sự

On-device (2025–26)

- **Apple Foundation Models** (∼3B, on-device, offline, free)
- **Chrome Gemini Nano** / Prompt API (Chrome 138, không gửi data đi)
- **MS Phi Silica** (NPU, Copilot+ PC)
- **WebLLM / transformers.js** (WebGPU trong browser)

Edge

Cloudflare Workers AI: inference trên GPU ở 200+ thành phố, pay-per-call, OpenAI-SDK compatible.

Đánh đổi: privacy + offline + cost 0, nhưng năng lực ∼1–3B, tốn pin, và **cold load** weights lần đầu.

Hybrid (mẫu hay nhất)

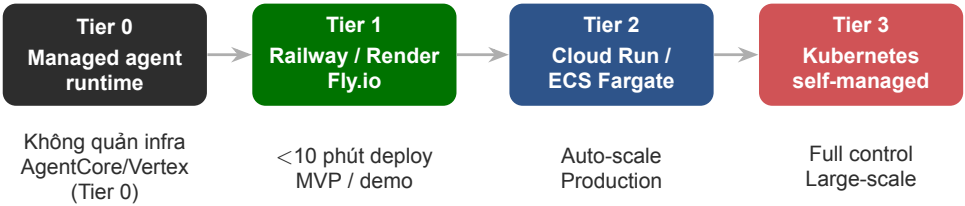
Model nhỏ on-device lo việc thường → **escalate** lên model lớn trên cloud khi khó (VD Apple: ∼3B → Private Cloud Compute). Tiêu chí định tuyến thường *không công bố*.

Cloud Deployment Options

06

Không có 1 platform đúng cho mọi trường hợp — và với agent, trục quan trọng nhất là **request timeout**, không phải giá

4 Tier Deployment (2026)



Cho khoá học

Bắt đầu Tier 1 (Railway/Render). Hiểu flow deploy trước, migrate lên Tier 2/3 khi business cần, hoặc Tier 0 nếu muốn bỏ qua việc quản hạ tầng.

So Sánh Platform — Theo Trục Timeout

Platform	Max request/runtime	Scale-to-0	GPU	Agent fit
Railway	15 phút / ∞ private	Không	Không	OK (route nội bộ)
Render	$\sim$100 phút **(?)**	Free tier	Không	OK
Fly.io	60s idle (stream reset)	Có	Có	OK + streaming
Cloud Run	**60 phút**	Có (cả GPU)	L4	Mạnh
AWS App Runner	$\sim$120s	Provisioned	Không	Deprecated
ECS Fargate	**Không giới hạn**	Không	—	Mạnh (always-on)
Modal	**24h** (mặc định **5 phút**)	Có (snapshot)	H100/A100	Mạnh (GPU)
Vercel functions	5 phút Hobby · 800s GA (Pro)	Có	Không	Kém

Lưu ý: Cập nhật 2026: Railway **bỏ free tier** (giờ $5 trial credit); AWS App Runner **deprecated** (Mar 2026) $\rightarrow$ ECS Express Mode; Cloud Run GPU **GA** (6/2025, scale-to-zero, $\sim$5s start). Con số $\sim$100 phút của Render **không có trong docs chính thức** (chỉ trong bài so sánh marketing) — đừng thiết kế dựa vào nó.

Serverless Functions — Tại Sao Không Hợp Agent

VINUNIVERSITY

Vercel / Lambda functions

- Hard cap **5 phút** (Hobby/mặc định); GA 800s, beta 1800s → rồi **504**
- Body cap 4.5 MB
- Stateless — mất context giữa các invoke

Container-based hợp hơn

- Giữ được connection cho streaming dài
- Min-instance giữ ấm, tránh cold start

Lưu ý: Câu cũ "serverless cold start 5–15s" giờ **lỗi thời**: Cloud Run GPU start $\sim$5s, Modal snapshot nhanh $\sim 10\times$. Vấn đề thật của serverless với agent là **timeout cap**, không phải cold start.

Railway — Deploy Trong 5 Phút

Các bước

1. Kết nối GitHub repo
2. Railway tự detect Dockerfile (builder Railpack 2026)
3. Set environment variables
4. Click Deploy
5. Nhận public URL

Tại sao chọn cho lab

- Auto-detect Dockerfile
- Environment variables UI
- Custom domain + SSL miễn phí
- Logs real-time
- $5 trial credit (đủ cho lab)

Lưu ý: **Hết free tier:** Railway tính theo usage; new user có $5 trial credit (không cần thẻ) **hết hạn sau 30 ngày**. Hết trial → về gói Free: chỉ $1 **credit/tháng** (không cộng dồn), và Railway **xoá volume** của tài khoản trial 30 ngày sau đó → *sao lưu trước khi hết hạn*. Bind đúng `0.0.0.0:$PORT`.

Khi Request/Response Không Còn Là Mô Hình

Chạy dài, không qua HTTP

- **Cloud Run Worker Pools** (GA 4/2026): instance chạy dài pull từ queue — **không có trần request timeout**. Cột "60 phút" *không áp dụng*.
- **Vercel Workflows**: durable execution — resume đúng điểm cũ, sống qua deploy/crash.
- **OpenAI Responses API** background mode.

Trần cũ vẫn còn nguyên

- **AWS Lambda**: trần cứng vẫn **900s**. API Gateway **29s** chặt hơn, nằm *phía trước* nó.
- Lambda response streaming: **native chỉ cho Node.js** — Python **không có**.

Ý chính

2026 platform mở lối thoát ở tầng hạ tầng: **bỏ hẳn mô hình request/response**. Chọn platform giờ là chọn **mode**.

Managed Agent Runtimes (Tier 0)

07

Danh mục mới hẳn của 2025–26: deploy agent mà **không phải quản container** — runtime, memory, identity đều managed

Tier 0 — Bạn Chỉ Mang Agent, Cloud Lo Phần Còn Lại

Bạn không viết Dockerfile, không lo scaling. Platform cấp sẵn runtime + session + memory + identity, tính theo tiêu thụ.

Sản phẩm	GA	Điểm nhấn
AWS Bedrock AgentCore	Oct 2025	**8 giờ/session**, microVM cô lập mỗi session
Agent Runtime (tên mới của Vertex AI Agent Engine)	Mar 2025	Sessions + **Memory Bank** + Code Execution + Example Store
Azure AI Foundry Agent	May 2025	No-code + hosted; **miễn phí service** (trả token)
OpenAI AgentKit	Oct 2025	**Agent Builder đóng 30/11/2026** → Agents SDK; ChatKit vẫn còn

Đặc điểm chung

Framework-agnostic (LangGraph, CrewAI, ADK...), hỗ trợ MCP/A2A (A2A nay là chuẩn **v1.0** do **Linux Foundation** quản, 150+ tổ chức), session isolation. *Cách tính tiền: slide sau.*

Framework vs Runtime — Hai Tầng Bạn Đang Chọn

VINUNIVERSITY

"Framework-agnostic" ở slide trước nghĩa là gì? Mọi hãng lớn ship **hai tầng riêng biệt** — và bạn chọn từng tầng độc lập.

Hãng	**Tầng framework** (bạn viết agent)	**Tầng managed runtime** (họ chạy)
Anthropic	Claude Agent SDK (Python/TS)	Claude Managed Agents (public beta)
OpenAI	Agents SDK	AgentKit; ChatKit là surface UI
Google	ADK (5 ngôn ngữ, OSS)	**Agent Runtime** (tên mới của Vertex AI Agent Engine)
AWS	Strands Agents (Apache-2.0)	Bedrock AgentCore
Microsoft	Agent Framework (AutoGen + Semantic Kernel)	Azure AI Foundry Agent Service
LangChain	**deepagents** (MIT)	LangSmith Deployment · Managed Deep Agents

Vì sao phải tách

Framework quyết định **code bạn viết**; runtime quyết định **ai bị đánh thức lúc 3h sáng**. Đổi framework = refactor. Đổi runtime = re-deploy. So sánh nhầm tầng là lỗi phổ biến nhất khi đọc tài liệu vendor.

Session Là Đơn Vị Vận Hành Của Tier 0

Ở Tier 0, **request** không còn là đơn vị — **session** mới là. Và session có trần riêng.

Trần session từng platform

- **AWS AgentCore**: tối đa **8 giờ**, chết khi **idle 15 phút**; mỗi session một microVM riêng, huỷ = wipe sạch.
- **Modal Sandbox**: mặc định **5 phút** — phải `timeout=` mới lên 24h.
- **Claude Managed Agents**: session tự chủ hàng giờ, *không công bố trần cứng*.

Khi cần vượt trần

- Modal **Filesystem Snapshot** → restore vào Sandbox mới.
- AgentCore **managed session storage** giữ state qua session.
- Nguyên tắc cũ vẫn đúng: state nằm **ngoài** runtime thì trần nào cũng vượt được.

Lưu ý: **Idle timeout là bẫy chi phí lẫn bẫy đúng-đắn:** agent chờ human approval 20 phút → session chết, state mất nếu chưa externalize.

Tier 0 Tính Tiền Thế Nào — Trục Chi Phí Thứ Ba

Tier 0 thêm **đồng hồ thứ hai** bên cạnh token: **thời gian thực** của session.

Platform	Đồng hồ tính tiền
AWS AgentCore	$0,0895/vCPU-giờ + $0,00945/GB-giờ — chỉ tính compute **active**; idle & I/O-wait **miễn phí**. Gateway $0,005/1.000 invocation
Claude Managed Agents	Giá token chuẩn **+ $0,08/session-giờ**, chỉ tính khi status = *running*
Cloudflare	**Active CPU pricing** — chỉ tính chu kỳ CPU thực chạy; thời gian nằm chờ LLM **miễn phí**
Azure AI Foundry Agent	Service **không tính phí riêng** — chỉ trả cho model + tài nguyên nền

Rút ra

Agent là workload **idle-heavy** (phần lớn thời gian là đợi LLM). Chọn Tier 0 chỉ cần hỏi **một câu**: *đồng hồ có chạy khi agent đang chờ không?* (Tối ưu chi phí ở quy mô → Day 25.)

Runtime Cũng Là Dependency Bạn Không Kiểm Soát

VINUNIVERSITY

Bằng chứng trong chính bài này

- **OpenAI Agent Builder** đóng **30/11/2026** — vòng đời **13 tháng**. Lối thoát: Agents SDK.
- **OpenAI Evals** cùng lịch → trỏ sang **Promptfoo**.
- **AWS App Runner** → ECS Express Mode.
- **LangGraph Platform** → LangSmith Deployment: đổi tên cũng là rủi ro — docs/IaC của bạn trỏ tên cũ.

Ba câu hỏi trước khi chọn

- **Pin cái gì?** SDK version, API version, image base.
- **Thoát bằng đường nào?** Export được state/config không?
- **Mất bao lâu?** Ước lượng thật.

Lưu ý: Cùng câu hỏi bạn đã hỏi về **model**, giờ hỏi về **runtime**: *pin cái gì, thoát bằng đường nào, mất bao lâu?*

Tier 0 vs Tự Deploy — Khi Nào Chọn Gì

Chọn Tier 0 khi

- Muốn ship nhanh, không có team infra
- Cần session isolation + memory sẵn
- Agent chạy rất lâu (AgentCore 8h)
- Đã ở sẵn hệ sinh thái AWS/GCP/Azure

Tự deploy (Tier 1–3) khi

- Cần kiểm soát đầy đủ stack
- Tránh vendor lock-in
- Tối ưu cost ở quy mô lớn
- Yêu cầu compliance/networking riêng

Lưu ý: Cho lab 12, ta vẫn **tự containerize + deploy** (Tier 1) để hiểu cơ chế. Tier 0 là lựa chọn production khi không muốn quản hạ tầng — biết là đủ.

Deep Agents — Thang 3 Bậc Của Chính LangChain

LangChain mô tả sản phẩm của họ như một **thang**, không phải lựa chọn nhị phân — và nó ánh xạ gần 1–1 vào Tier 0–3 ở đầu mục này.

Bậc	Bạn sở hữu	Đánh đổi
OSS **deepagents** (MIT)	Toàn bộ hosting	Kiểm soát tối đa; **tự cấu hình persistence**
LangSmith Deployment	Application + server code	Cho team cần route riêng, auth nâng cao, scale lớn
Managed Deep Agents	Chỉ agent code, tools, middleware, instructions	LangChain lo backend, store, checkpointer, memory, skills, sandbox, identity

Lưu ý: **Managed Deep Agents đang PRIVATE BETA**: vào bằng **waitlist**, chỉ chạy ở region **US** của LangSmith Cloud, **CLI-first**, docs ghi rõ "behavior may change before general availability", và **chưa công bố giá**. Biết là đủ — đừng thiết kế kiến trúc quanh nó.

Deploy Deep Agents — Bốn Quyết Định Hạ Tầng

VINUNIVERSITY

Filesystem backend = quyết định infra

- **StateBackend** (mặc định): theo thread, **không** chia sẻ cross-thread
- **StoreBackend** / **CompositeBackend**: chia sẻ cross-thread
- **FilesystemBackend** và **LocalShellBackend** truy cập thẳng host — docs: **đừng dùng** trong agent đã deploy

Sandbox & secret

- **Thread-scoped**: sandbox mới mỗi hội thoại, dọn khi hết TTL
- **Assistant-scoped**: dùng chung → tích tụ file & package → **phải** đặt TTL
- **Auth proxy** chèn credential vào request đi ra → secret **không vào** sandbox

Lưu ý: **Hai bẫy còn lại:** (1) LangSmith Deployment **tự** cấu hình persistent checkpointer — self-host thì bạn phải tự làm, đây là delta local→prod lớn nhất. (2) **Shared memory** là vector **prompt injection** — scope namespace theo user (Day 11).

“Open Source” Không Có Nghĩa Là Tự Host Được Miễn Phí

Bạn tự host để tránh lock-in. Nhưng licence của **server** thường khác licence của **library**.

Thành phần	Giấy phép	Ràng buộc khi self-host
LangGraph (library lõi)	MIT	Tự do
`langgraph-api` (server)	**Elastic-2.0**	Cần license key + egress tới `beacon.langchain.com` (có chế độ air-gapped)
n8n	Sustainable Use License	Chỉ nội bộ / phi thương mại
Dify	Apache 2.0 **sửa đổi**	Cấm chạy multi-tenant SaaS; cấm gỡ logo
Claude Code	**Proprietary**	Anthropic Commercial ToS — **không phải open source**
Codex CLI · opencode · goose	Apache-2.0 · MIT · Apache-2.0	Không ràng buộc
Crush	FSL-1.1-MIT	Đọc được mã nhưng **cấm dùng cạnh tranh**; MIT sau 2 năm

Lưu ý: **Ba câu hỏi khi thấy chữ “open source”:** licence của **server** có giống licence của **library**? Có thư mục `ee/`? Có phải **gọi về nhà** để xác thực licence? — Và OSS **có thể đóng lại**: Daytona đóng mã lớp sandbox ngày **11/6/2026**.

Hosting MCP Servers

08

Day 9 dạy agent gọi MCP tools. Khi MCP server cần chạy *re-mote* cho nhiều client, nó cũng là một service phải deploy — và phải bảo mật đúng

stdio vs Streamable HTTP — Hai Transport Của MCP

stdio (local)

Client **spawn server làm subprocess**, nói chuyện qua stdin/stdout.

- Chạy cùng máy với agent
- Không cần OAuth — lấy credential từ env
- Hợp dev / desktop

Streamable HTTP (remote)

Một endpoint (vd `/mcp`), POST+GET, SSE *bên trong*.

- Process độc lập, phục vụ nhiều client
- **Thay thế** transport HTTP+SSE cũ (2024-11-05)
- Phải host + bảo mật như API thật

Lưu ý: Remote MCP server (spec **2026-07-28**) phải dùng **OAuth 2.1**. stdio thì KHÔNG cần. *Ba cái bẫy spec gọi tên: slide sau.*

MCP 2026-07-28 — Protocol Trở Thành Stateless

Thay đổi lớn nhất

- Ba bản: 2025-06-18 → 2025-11-25 → **2026-07-28**.
- **"Make MCP stateless"**: bỏ handshake `initialize` + header `Mcp-Session-Id`.
- Transport **HTTP+SSE** cũ **deprecated** (12 tháng chuyển tiếp).
- Header `Mcp-Method`/`Mcp-Name`: router khỏi parse JSON body.

Hệ quả deploy

Mọi request rơi vào **bất kỳ instance nào** sau LB round-robin thường: không sticky session, không session store, scale-to-zero thoải mái.

Đúng bài học mục Statefulness: externalize state → tự do scale.

Lưu ý chuyển tiếp

Server đã deploy vẫn phải **tương thích ngược** với client bản 2025 trong suốt giai đoạn chuyển tiếp — đừng xoá code cũ ngay.

OAuth 2.1 Cho MCP — Ba Cái Bẫy Spec Gọi Tên

VINUNIVERSITY

Bẫy 1 — Token passthrough

Server **MUST NOT** nhận token không được cấp riêng cho nó. Spec **cấm rõ ràng**.

Vì sao nguy hiểm: nó **vô hiệu hoá rate limiting và audit trail** ở service phía sau.

Bẫy 2 — Confused deputy

Proxy phải giữ registry `client_id` **đã duyệt theo từng user**, kiểm tra trước mỗi flow.

Xác thực đúng *user* $\neq$ uỷ quyền cho *client*.

Bẫy 3 — Discovery đã đổi

Từ 2025-11-25 theo **RFC 9728**: `WWW-Authenticate` nay **tuỳ chọn**, fallback về `.well-known`.

Siết thêm ở bản 2026-07-28

- Validate `iss` theo RFC 9207
- Khai `application_type` lúc đăng ký
- **DCR** đang bị khai tử $\rightarrow$ **CIMD**

Lưu ý: stdio **không** cần OAuth. Nhưng khoảnh khắc bạn đưa server ra remote, nó là **một API công khai với đầy đủ nghĩa vụ**.

Server MCP Là API Thật — Ba CVE Đã Chứng Minh

CVE	Bài học deploy
CVE-2026-33032 nginx-ui, CVSS **9,8** *đang bị khai thác*	`/mcp` có `AuthRequired()`, `/mcp_message` **thì không** → 12 tool không cần credential, chiếm server trong **2 request**. → **Xác thực MỌI route.** Lỗi deploy kinh điển, không phải lỗi protocol.
CVE-2025-6514 `mcp-remote`, 9,6	Command injection trong chính **OAuth proxy**: chỉ cần kết nối tới server độc hại là RCE trên máy client. → Lớp proxy **cũng** là attack surface.
CVE-2025-68143/4/5 Git MCP server (Anthropic)	Path traversal + argument injection; `git_init` bị gỡ hẳn. → Ngay cả server tham chiếu của *người tạo protocol* cũng có lỗ hổng.

Lưu ý: Rất nhiều MCP server chỉ là **lớp bọc mỏng quanh một CLI**. Deploy remote = phơi CLI đó ra Internet → container, non-root, egress allowlist.

Deploy MCP Ở Đâu — Registry, Gateway, Runtime

Lớp	Vai trò	Công cụ 2026
Registry	Để *tìm*	**Official MCP Registry**: `server.json` + namespace reverse-DNS. **Chỉ là metadata — KHÔNG phải hosting**; giải quyết *discovery*, không giải quyết *trust*
Gateway	Để *gom* & *kiểm soát*	**Docker MCP Gateway** (mỗi server một container cô lập, secret tập trung); **AWS AgentCore Gateway** (gom Lambda/REST/MCP về một endpoint có governance)
Runtime	Để *chạy*	**Cloudflare Workers** + OAuth 2.1 qua `workers-oauth-provider`; hoặc chính container bạn đã học ở mục Docker

Mẫu 2026

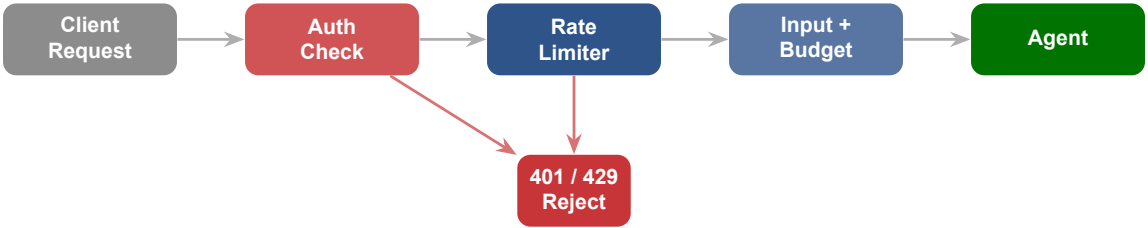
Registry để tìm → **Gateway để gom + kiểm soát** → **Runtime để chạy.** Đừng để mỗi agent tự cắm thẳng vào 20 server rời rạc.

API Gateway & Security

09

Agent trên cloud cần lớp bảo vệ trước khi request đến logic — authentication, rate limiting, và **cost protection** (nơi nhiều startup đã “cháy túi”)

API Gateway Architecture



Nguyên tắc

Mỗi request phải qua **auth** → **rate limit** → **validate + budget check** trước khi agent xử lý. Reject sớm = tiết kiệm tokens và tiền.

Authentication Patterns

API Key

Đơn giản nhất.

Header: `X-API-Key`

Dùng khi: internal, MVP, B2B (M2M)

JWT Token

Stateless auth.

Bearer token + expiry

Dùng khi: user-facing app, microservices

OAuth 2.1

Delegated auth + PKCE.

Chuẩn cho MCP/agent remote

Dùng khi: platform, remote MCP

Lưu ý: Cho MVP: **API key** là đủ. Đừng over-engineer auth trước khi có user thật. Nhưng nếu host **remote MCP server**, OAuth 2.1 là bắt buộc theo spec.

Cost Protection — Đừng Để Agent Đốt Hết Tiền

VINUNIVERSITY

Rủi ro (có thật)

- **$47.000 trong 11 ngày:** 4 agent retry vô hạn, có log nhưng không có hard limit
- **$96.000 Vercel:** app Cara tăng 100k→900k user trong vài ngày
- Prompt injection → token explosion
- API key bị lộ → hacker xài

Bảo vệ

- **Pre-call admission control:** check budget còn lại *trước khi* gọi LLM, từ chối nếu vượt
- **Per-tenant budget:** key theo (tenant, workload, model)
- **Rate limiting:** token bucket (rate + burst)
- **Circuit breaker:** tắt khi anomaly

Lưu ý: **Bẫy lớn:** "*spend alert*" chỉ **thông báo, KHÔNG chặn**. Hard cap là tính năng riêng, phải **bật thủ công**. Alert của provider là *phản ứng sau* → vẫn phải tự build admission control *chặn trước*.

Security basics

- **HTTPS:** Railway/Render cấp SSL tự động
- **CORS:** chỉ là kiểm soát *trình duyệt* — KHÔNG phải authz, API vẫn cần auth
- **Secrets:** dùng secret manager (Doppler, Infisical...) thay `.env` — có rotation + audit
- **GitHub:** push protection mặc định; key lộ bị auto-revoke

OWASP LLM Top 10 (2026)

- LLM01 Prompt Injection
- LLM02 Sensitive Info Disclosure
- LLM06 Excessive Agency
- **LLM10 Unbounded Consumption** (cost/DoS)

→ Agent có tool: **Agentic Apps 2026** — **ASI03** (Identity) + **ASI04** (Supply Chain). *Đầy đủ: Day 11*

Lưu ý: Kiểm tra ngay: `.env` có trong `.gitignore`? Key có lỡ commit lên GitHub? Nếu có → **revoke và tạo key mới ngay lập tức**.

Dữ Liệu Chạy Ở Đâu — Residency, ZDR & Compliance

Gateway lo auth, rate limit, cost. Còn một trục nữa nhiều team chỉ phát hiện lúc ký hợp đồng: **dữ liệu nằm ở đâu và tồn tại bao lâu**.

OpenAI

- **11 region** lưu trữ at-rest, nhưng chỉ **US / EU / UAE** xử lý inference trong vùng
- Bật theo project bằng **prefix domain** (`eu.api.openai.com`); **phụ phí 10%**
- **ZDR** loại nội dung khỏi log, **ép** `store=false`, phải duyệt trước

Anthropic

- **Claude Managed Agents KHÔNG đủ điều kiện ZDR lẫn HIPAA BAA** — vì nó *cố ý* lưu history, sandbox state, output ở server
- **Claude Platform on AWS** $\neq$ Bedrock: hạ tầng do **Anthropic** vận hành; ZDR xin được

Lưu ý: **Việt Nam không nằm trong danh sách region** $\rightarrow$ mặc định dữ liệu **rời lãnh thổ**. Nếu hồ sơ pháp lý buộc dữ liệu ở lại, lựa chọn còn lại là **self-host** — chính là quyết định ở mục sau.

Hard Cap Thật Sự Nằm Ở Đâu?

Nhà cung cấp	Hard cap native?	Cơ chế	Bẫy
OpenAI	Có, **phải bật thủ công**	Hard Spend Limit *ở cả* org lẫn project, trả **429**	"Spend alert" và trường monthly-budget cũ **chỉ báo, không chặn**
Anthropic	**Có, thật sự cứng**	Trần theo tier ($500 / $1.000 / $200.000); chạm trần → **API tạm dừng tới tháng sau**	Có spend limit **theo Workspace**, rút từ hạn mức chung của org
AWS Bedrock	**KHÔNG**	Tự ráp: Budgets + CloudWatch → Lambda thu hồi IAM	Circuit breaker **phản ứng sau**, không phải admission control
LiteLLM	Có **theo thiết kế**	Virtual key `max_budget` → 429 `BudgetExceededError`	Đã có issue **bypass hoàn toàn** ở một bản release

Kết luận

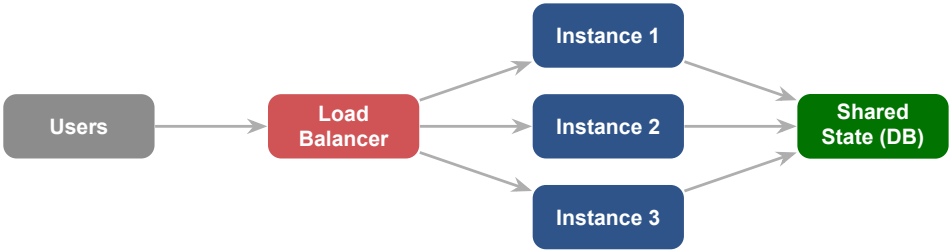
Enforcement của provider **không đồng nhất** và không phải lúc nào cũng hoạt động → admission control phía bạn là lớp duy nhất bạn kiểm soát. *"Cứng theo thiết kế"* ≠ *"cứng ở version bạn đang deploy" — hãy tự test.*

Scaling & Reliability

10

Agent MVP không cần Kubernetes — nhưng cần hiểu cơ bản về scaling và reliability để hệ thống không chết khi có nhiều user hơn

Horizontal Scaling — Scale Theo Concurrency, Không Phải CPU



Lưu ý: Agent là **I/O-bound**: một instance có thể đầy request đang chờ LLM mà CPU vẫn thấp. → Autoscale theo **concurrency / queue depth** (Knative, KEDA theo tín hiệu vLLM `num_requests_running`, Ray Serve `target_ongoing_requests`), không theo CPU. Điều kiện tiên quyết: agent **stateless**, state ở DB/Redis.

Health Checks — 3 Loại Probe

3 loại probe

- **Liveness** — “còn sống?” Fail → **restart** container (`GET /health`)
- **Readiness** — “sẵn sàng nhận request?” Fail → **gỡ khỏi LB**, KHÔNG restart
- **Startup** — cho boot chậm (load weights); gate 2 probe kia đến khi pass

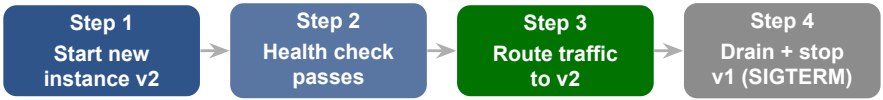
Health endpoint mẫu

`GET /health` trả về:

- status: “ok” / “degraded”
- uptime: seconds
- version: app version
- dependencies: DB, vector store, LLM reachable?

Lưu ý: **Liveness** restart, **readiness** chỉ ngắt traffic — cấu hình nhầm readiness thành liveness = restart loop vô ích.

Zero-Downtime Deploy & Graceful Shutdown

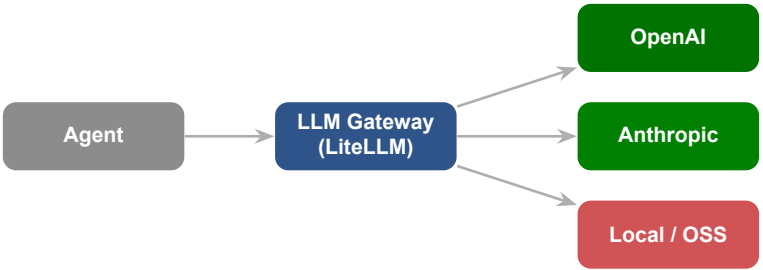


Lưu ý: **Graceful shutdown cho agent:** khi nhận SIGTERM, phải **drain** request đang chạy dở (một agent turn có thể dài). Đặt `terminationGracePeriodSeconds` **lớn hơn worst-case agent turn**, nếu không request bị cắt giữa chừng.

Nâng cao

Rainbow deployment (Anthropic): dịch traffic dần sang version mới nhưng **giữ cả hai cùng chạy**, để không cắt ngang agent đang chạy dở — vì agent là hệ thống stateful chạy gần như liên tục. Argo Rollouts: shift traffic + AnalysisRun tự rollback. Railway/Render hỗ trợ rolling sẵn.

LLM Gateway / Router — Một Cửa, Nhiều Provider



- **1 endpoint, format OpenAI** cho hàng trăm model — **LiteLLM** (tự host, không markup) vs **OpenRouter** (managed, phí nạp credit 5,5%)
- **Fallback & failover:** model lỗi / rate-limit → tự nhảy nhóm dự phòng; load-balance nhiều key, retry/backoff, cost tracking một chỗ

Vì sao cần

Router **vượt trần** rate-limit của vendor bằng nhiều key/nhiều provider, và sống sót khi một provider hỏng.

Cache Để Cắt Cost & Latency

VINUNIVERSITY

Prompt caching (provider)

Tái dùng KV-cache của **prefix** chung (system prompt, RAG context).

- Anthropic: cache read $\sim 0.1\times$ ($\sim$90% rẻ hơn), cần khai báo breakpoint; **ghi** cache $1.25\times$–$2\times$
- OpenAI: **tự động** với prefix $\geq$1024 token, cache read $\sim 0.1\times$ ($\sim$**90%** rẻ hơn); từ GPT-5.6 có thêm **phí ghi cache** $1.25\times$

Semantic cache (của bạn)

Trả lại câu trả lời cũ cho câu hỏi **tương tự về nghĩa** (so embedding).

- GPTCache: lưu embedding query trong Redis
- Tránh gọi LLM lặp $\rightarrow$ cắt cả cost lẫn latency
- Cẩn thận: trả lời cũ có thể lỗi thời

Lưu ý: **Prompt cache** so khớp prefix từng byte; **semantic cache** so khớp ý nghĩa — mạnh hơn nhưng có thể trả câu cũ đã lỗi thời.

Deploy Model — Gọi API Hay Tự Phục Vụ?

Trước hết: có tự host model không? Nếu có, phần sau giải thích **vì sao cost & latency lại như vậy** — một sợi chỉ đỏ duy nhất: *giữ GPU bận, đừng phí KV cache*

Model: Gọi API Hay Tự Phục Vụ? — Quyết Định Deploy Thứ Hai

VINUNIVERSITY

Bốn lực đẩy sang tự host

- **Residency / compliance**: dữ liệu không được rời lãnh thổ hoặc VPC
- **Sở hữu adapter** fine-tune (Bedrock **Custom Model Import** làm mờ ranh giới)
- **Kinh tế theo volume**: chỉ thắng khi GPU đủ bận (hoà vốn: Day 25); **trần năng lực** không còn là rào — slide sau

Ba lực giữ ở hosted API

- **Không có GPU-hour rảnh**: hosted trả theo token; tự host trả theo **giờ**, kể cả khi rảnh
- **Bậc mua năng lực**: **Fast mode** (2×), **Scale Tier** (SLA 99,9%)
- **Ngược lại:** Azure **PTU** tính **theo giờ dù không có token** — đúng cái Tier 0 tránh được

Ba lớp, không phải hai

Hosted API → **managed self-hosting** (bạn mang weight, họ lo serving stack) → tự dựng. Câu hỏi không phải "model nào giỏi hơn" (Day 14) mà "**tôi có muốn vận hành thêm một service GPU không**".

Nếu Tự Host: Chọn Model Mở Nào, Engine Nào

Model mở đáng deploy (8/2026)

- `gpt-oss-120b` — 117B tổng / **5,1B active** (MoE), Apache-2.0, vừa **một GPU 80GB** nhờ MXFP4
- `gpt-oss-20b` — chạy trong **16GB**: tầng phần cứng phổ thông
- **Qwen3-235B-A22B** — 235B/22B, Apache-2.0
- **DeepSeek-V3.2-Exp** — 685B, MIT
- **Kimi K2** — 1T/32B, Modified MIT, thiết kế cho agentic tool-use

*MoE: VRAM quyết bởi **active params**, không phải tổng.*

Engine chọn theo hình dạng tải

- **vLLM** — mặc định đa phần cứng (NVIDIA/ROCm/Intel/TPU), server OpenAI-compatible
- **SGLang** — khi prefix được dùng lại nhiều (RadixAttention) → rất đúng với agent
- **TensorRT-LLM** — đỉnh trên NVIDIA, đổi lại phải compile engine riêng
- **llama.cpp + GGUF** — CPU/edge/consumer, quantize 1,5–8 bit

Quantization không thay nhau: GGUF (CPU/edge) ≠ AWQ/GPTQ (GPU) ≠ FP8 (H100/Blackwell).

Lưu ý: Ship fine-tune bằng **multi-LoRA trên một base**, không phải một GPU cho mỗi adapter. *Cơ chế bên trong engine, benchmark, sharding đa GPU: Day 20.*

Tự Host = Bạn Có Thêm Một Service Phải Deploy

Vận hành đổi luật

- **Cold start không còn tính bằng giây**: nạp weight hàng chục GB → **phút**
- **Readiness probe** phải đợi weight nạp xong, không chỉ đợi port mở
- Autoscale theo **queue depth**, không theo CPU
- **GPU rảnh vẫn tính tiền** — ngược hẳn đồng hồ idle-free của Tier 0

Artifact deploy có thêm trục thứ tư

Nhớ ba artifact rollback ở mục CI/CD — **image · prompt · model ID**? Tự host thêm: **weight** (repo + revision) · **định dạng/quantization** · **phiên bản engine**. Hỏi trước khi deploy: *"rollback" ở đây nghĩa là rollback **cái nào**?*

Lưu ý: Tự host không phải là "bỏ vendor" — là **đổi vendor lock-in lấy công việc vận hành**. Mọi mục hôm nay (health check, scaling, rollback, cost guard) đều phải làm lại cho service GPU này.

Sợi Chỉ Đỏ: Giữ GPU Bận + Continuous Batching

Static batching

Cả batch chờ request **dài nhất** xong → GPU ngồi không khi các request ngắn đã xong.

Continuous (in-flight) batching

Lên lịch lại batch **mỗi bước decode**: request xong thì thay ngay request mới vào.

Tác động

Orca (OSDI'22) giới thiệu iteration-level scheduling, báo cáo tới **36.9**× throughput so với baseline ở cùng latency. Đây là "unlock" lớn nhất của GPU utilization khi serve LLM.

Lưu ý: Mọi kỹ thuật sau phục vụ một ý: **không phí KV cache, GPU không bao giờ rảnh** — vì **GPU-hour** quyết định cost serve agent.

PagedAttention — Quản KV Cache Như OS Quản RAM

Mỗi token sinh ra cần lưu Key/Value (KV cache). Cấp phát liền mạch → phân mảnh, phí bộ nhớ. **PagedAttention** (vLLM, SOSP'23) chia KV cache thành **block cố định, không liền mạch** — y như virtual memory paging của hệ điều hành.

- Gần **zero** lãng phí KV → batch lớn hơn trong cùng VRAM
- Chia sẻ KV giữa request (parallel sampling, beam) qua copy-on-write
- vLLM báo cáo **2–4**× throughput so với hệ trước ở cùng latency

Vì sao bạn quan tâm

KV cache là lý do context dài **tốn bộ nhớ** và vì sao prompt caching (slide trước) tiết kiệm được — nó tái dùng đúng những block KV này.

Prefix Caching + Speculative Decoding

VINUNIVERSITY

Prefix / prompt caching

Prefill của prefix chung tính **một lần**, dùng lại cho nhiều request.

- Prefill của prefix tính **một lần** rồi tái dùng → đặt phần **tĩnh** (system, few-shot, RAG) lên **đầu** prompt. *(Giá cache: mục Cache ở slide trước.)*

Speculative decoding

Model **nháp** nhỏ đề xuất 5–8 token, model **đích** xác minh song song.

- Tận dụng GPU đang rảnh, **không đổi** phân phối output
- $\sim$2–3$\times$ giảm latency; EAGLE-3 báo cáo tới $\sim$4.8$\times$ (Llama-3.3-70B)

Liên hệ thực tế

Hai kỹ thuật này là lý do cùng một câu hỏi gửi lần 2 (cache hit) **rẻ và nhanh hơn** hẳn — thiết kế prompt để tận dụng.

Disaggregated Prefill / Decode



Prefill (đọc cả prompt, song song $\rightarrow$ **compute-bound**) và decode (sinh từng token $\rightarrow$ **memory-bandwidth-bound**) có hồ sơ tài nguyên khác nhau $\rightarrow$ tách ra 2 pool GPU **scale độc lập**.

- DistServe (OSDI'24): tới **7.4**$\times$ nhiều request hơn trong cùng SLO
- Mooncake (Kimi, FAST'25): KV-cache-centric, $>$100B token/ngày
- NVIDIA Dynamo (GTC 3/2025): prefill/decode là first-class, KV-aware router

CI/CD & Eval Gates

12

Deploy bằng tay sẽ quên bước, sẽ sai. Tự động hoá build→push→deploy — và thêm lớp **eval gate** riêng của AI: chặn deploy nếu chất lượng tụt

Pipeline: Build → Push → Deploy → Eval Gate


```yaml
# .github/workflows/deploy.yml
steps:
  - uses: actions/checkout@v4
  # build + push image to GHCR (needs packages: write)
  - uses: docker/build-push-action@v6
  # scan image for CVEs, fail the build on high severity
  - uses: aquasecurity/trivy-action@master
  - run: promptfoo eval --fail-on-error   # <- EVAL GATE
  - run: railway up                       # deploy iff evals pass
```

Eval gate — lớp đặc trưng của AI

Agent **non-deterministic** → không thể chỉ dựa unit test `exit 0`. Gate deploy theo **điểm eval** $\geq$ **ngưỡng** (promptfoo / DeepEval / Braintrust). Đây là cầu nối tới Day 14 (Evaluation).

Eval Gate Trong Thực Tế — Bốn Công Cụ, Một Khuôn

Công cụ	Cơ chế gate	Kết quả trên PR
promptfoo	`promptfoo eval --fail-on-error`; chặt hơn: fail khi `stats.failures > 0`	`promptfoo-action` comment pass/fail + link viewer
DeepEval	`assert_test()` (**không phải** `evaluate()`) — raise khi score $<$ threshold	pytest-native, fail như test thường
Langfuse	`experiment-action` chạy trên dataset có version	Comment kết quả lên PR
Braintrust	`eval-action` chặn merge khi dưới ngưỡng	**Diff view** so với run của nhánh baseline

Lưu ý: Đừng pin `temperature=0`: Anthropic deprecate `temperature`/`top_p`/`top_k` từ **Opus 4.7** trở đi — set khác mặc định trả **400**. Mọi harness đang pin `temperature=0` sẽ **gãy thẳng**.

Mẫu đã hội tụ

Cả bốn công cụ đi tới cùng một chỗ: so với **run của nhánh baseline**, không chỉ so với một ngưỡng tuyệt đối — ngưỡng tuyệt đối không phát hiện được "tụt 4 điểm nhưng vẫn trên ngưỡng". *Phương pháp đánh giá → Day 14; quan sát production → Day 13.*

Shadow → Canary → 100% — Ramp Cho Agent



- **Shadow (mirror):** nhân đôi traffic sang version mới, **không bao giờ** trả output cho user. Rủi ro user $= 0$.
- **Canary:** định tuyến **một % traffic thật** *và có* trả output — bán kính thiệt hại giới hạn.
- **Auto-rollback** là primitive có thật: Argo Rollouts `AnalysisRun` query Prometheus theo lịch, tự rollback khi metric fail.

Lưu ý: **Thuế riêng của agent:** shadow chạy *cả hai version cho mọi request* → **nhân đôi hoá đơn token**. Với agent: shadow **một mẫu 5–10%**, không phải 100%. Và shadow chỉ an toàn khi tool call của version mới **bị chặn side effect** — nếu không, "không trả output" vẫn **trừ tiền thật**.

Rollback Không Nhất Thiết Là Một Lần Deploy

VINUNIVERSITY

Ba artifact, ba vòng đời

Một "bản deploy" của agent thực ra gồm **ba thứ rollback độc lập**:

- Container **image**
- **Prompt version**
- **Model ID**

Ba đường rollback khác nhau — đừng gộp làm một.

Mẫu đã hội tụ ở 6 vendor

Prompt version là artifact **bất biến, content-addressed**; một **label di động** (`prod`) được trỏ lại để promote hoặc roll-back.

→ Rollback prompt/model xấu = **di chuyển một label**, tính bằng **giây**: không deploy, không chạy CI.

Giới hạn: label chỉ cứu prompt/config — code và image vẫn đi qua pipeline.

Lưu ý: Nếu prompt của bạn nằm trong `.py` và đi cùng image, bạn **không có** rollback nhanh — bạn có một lần deploy. **Tách prompt ra khỏi code là một quyết định deploy.**

Agent Là Một Job Trong CI — Hợp Đồng Tối Thiểu

VINUNIVERSITY

Hợp đồng gọi (callable contract)

- Exit `0` / khác `0` để script rẽ nhánh; `--output-format json` trả `total_cost_usd` mỗi lần chạy
- Chế độ **bare**: bỏ auto-discovery hook/skill/MCP → CI chạy **giống nhau mọi máy**
- Đừng parse transcript nội bộ — format **không ổn định**

Ai được kích hoạt, với quyền gì

- Gate **danh tính người kích hoạt** trước gate tool: write access + phải là người
- **Bẫy:** trigger `schedule` **không có tác giả** → *bỏ qua* check write-access
- Automation mode: **zero tool** mặc định; **OIDC federation** thay secret API key tĩnh trong repo

Lưu ý: Job agent có **hai trục chi phí độc lập** — compute-minute *và* token — phải chặn cứng cả hai (`--max-turns`, workflow timeout, concurrency limit). Và tách **pha**: cấp secret cho setup script rồi **gỡ trước khi** vòng lặp agent đọc nội dung không tin cậy.

Nâng Cao — Deploy Cấp Production (tùy chọn)

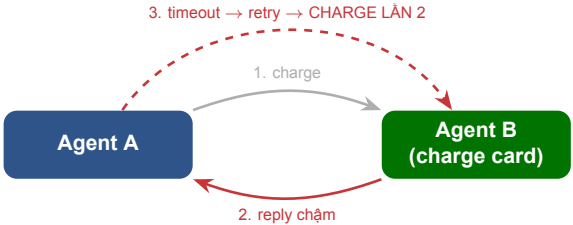
Đưa 1 agent lên URL an toàn mới là 20% dễ. 80% khó là khi agent **bền bỉ, lặp lại, và hành động không hoàn tác được** — phần này phác qua những vấn đề đó

Agent Có Nhiều "Hình Dạng" — Mỗi Cái Deploy Khác Nhau

Hình dạng	Trigger	Vòng đời	Scale	State
Chatbot đồng bộ	user request	giây (timeout)	per-request	session store
Cron / nền	scheduler	phút	thấp, định kỳ	job state
Batch	dataset/queue	dài, async	fan-out	per-item idempotent
Autonomous "chạy mãi"	loop liên tục	**vô hạn**	1 actor/goal	**phải sống qua restart**
Copilot nhúng	in-app event	giây	theo app	app/session
Ambient / sự kiện	webhook/event	bursts, ngủ lâu	scale-to-zero	bền qua giấc ngủ

Lưu ý: **State-durability là trục phân biệt chính.** "Autonomous chạy mãi" phá vỡ mô hình request/response → buộc dùng durable runtime. Multi-agent: tốn $\sim 15\times$ token; nguyên tắc vàng "**read thì song song được, write thì không**" — scale 1 agent trước, chỉ tách khi chạm trần thật.

Web app thường: lỗi thì retry. Nhưng agent có **side effect thật** (gửi mail, trừ tiền, đặt vé). Retry



mù = **làm hai lần**.

Lưu ý: Timeout **không phân biệt** được “thất bại” với “thành công nhưng reply chậm” → caller bắn lại side effect. Sự thật phũ phàng: **không có “exactly-once”** cho hành động ngoài hệ thống — chỉ có **at-least-once + consumer idempotent**.

Idempotency Key — Và Cái Bẫy Riêng Của Agent

Mẫu chuẩn (Stripe)

Client gửi header `Idempotency-Key` (UUID) kèm request mutating.

- Server lưu kết quả lần **đầu** theo key
- Retry cùng key → trả lại y nguyên (kể cả lỗi đã cache)
- Stripe là chuẩn *de-facto*, **không phải RFC** (IETF draft đã hết hạn)

Bẫy với LLM agent

LLM **không sinh lại tham số y hệt** cho cùng một ý định → hash nội dung thô **trượt**.

- Cần **dedup theo ngữ nghĩa / ý định**
- "Cùng intent", không phải "cùng bytes"
- Ai là trọng tài quyết 2 hành động là "một"?

Thực hành

Mọi tool call có side effect nên mang một idempotency key ổn định (vd `order_id`), và backend dedup bằng Redis `SET NX` + TTL.

Durable Execution — Ghi Lại Quyết Định, Đừng Chạy Lại Model

Agent crash ở bước 7 (sau 4 tool call). Chạy lại từ đầu = **trả tiền LLM lần nữa** + lặp tool đã ghi đĩa. Durable execution giải quyết bằng một mẹo tinh tế:

Journal + replay

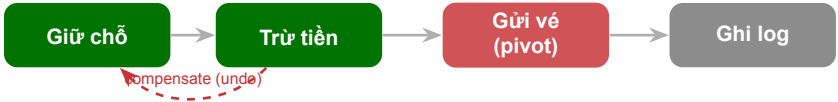
Temporal / Restate / Inngest: ghi **output của LLM** vào journal ở lần đầu; khi replay thì **đọc lại bản ghi**, **KHÔNG gọi lại model**.

- "Phần thông minh" không bao giờ chạy lại
- Bước đã xong được memoize, bỏ qua

Lưu ý: **LangGraph là ngoại lệ:** checkpoint ở mức **node**, không phải từng call → node chưa xong sẽ **chạy lại cả LLM call**. "Checkpoints ≠ durable execution".

Lưu ý: "Exactly-once" của các framework thực ra là "exactly-once *trên datastore của họ*". Hành động ra hệ thống ngoài vẫn cần idempotent. Câu hỏi hay: agent được **resume** có còn "suy nghĩ" không, hay chỉ đang **đọc lại** quá khứ?

Saga & Hành Động Không Thể Hoàn Tác



- **Saga:** mỗi bước có một bước **bù trừ** (undo) — không có rollback tự động, phải tự code
- Bước **không hoàn tác được** (gửi mail/vé) = **pivot**: đặt **cuối** + gate bằng **human approval**
- Vấn đề: agent tự chọn thứ tự hành động — runtime có nên **cấm** nó xếp việc bất khả hoàn trước pivot?

Lưu ý: HITL nghịch lý: Temporal cho agent **chờ 3 tuần tốn 0 compute** — nhưng giữ chỗ / báo giá / token **vẫn hỏng dần** ngoài đời. Resume một quyết định đã cũ = durability thành gánh nặng.

Security: Egress Là Điểm Kiểm Soát

Đây là mặt **hạ tầng** của an toàn (khác Day 11 — mặt hành vi). Tool của agent gọi ra ngoài = kênh rò rỉ.

Lethal trifecta (Willison)

Rò rỉ data cần 3 thứ **cùng lúc**:

- data riêng tư
- + nội dung không tin cậy
- + kênh gửi ra ngoài

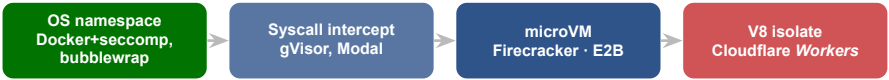
Bỏ một cái = chặn được.

4 sự cố 2025, cùng 1 cơ chế

EchoLeak · CamoLeak · GitLab Duo · AgentFlayer — đều **render ảnh/HTML tới URL kẻ tấn công** và đều fix bằng **chặn render / giới hạn egress**.

Lưu ý: Egress allowlist là phòng thủ mạnh nhất — nhưng **kênh rò rỉ thường là kênh tin cậy**: CamoLeak tuồn data qua chính proxy Camo của GitHub. Không thể allowlist khỏi nhà cung cấp bạn đang tin.

Sandbox & Bài Học “Fail-Open”



cô lập mạnh hơn → (startup chậm hơn, gần như ngược lại)

Lưu ý: **CVE-2025-66479 (Claude Code):** cấu hình `allowedDomains: []` (ý định *chặt nhất*) lại **fail-open** (mở toang egress) vì code check `length > 0`. Ý định an toàn nhất **không biểu diễn được**. → **Ngữ nghĩa mặc định CHÍNH LÀ thuộc tính bảo mật** (fail-closed, không phải fail-open).

Nhãn đúng 2026: `Cloudflare Sandboxes` (GA 13/4/2026) chạy trên **Containers**, không phải V8 isolate — V8 isolate chỉ đúng cho *Workers*.

Least-Privilege — Mỗi Agent Một Danh Tính

Rủi ro

- OWASP **LLM06 Excessive Agency**: quá nhiều quyền → hành động phá hoại
- **Confused deputy**: agent bị lừa dùng quyền của nó cho kẻ khác
- 1 “god-key” chung = 1 điểm sụp đổ

Phòng thủ

- Mỗi agent một **identity** riêng, scope hẹp
- Token **ngắn hạn**, theo từng task
- Audience-bound token (MCP OAuth 2.1)
- VD: Microsoft Entra Agent ID

Lưu ý: Khoảng cách thực tế: ∼91% tổ chức chạy agent ở production, nhưng chỉ ∼10% quản chúng **như một danh tính**. Per-agent identity ở quy mô lớn là bài toán vận hành chưa được giải tốt.

Danh Tính Cho Agent — SPIFFE Và Agent Identity

VINUNIVERSITY

Cơ chế đang thành chuẩn

Service account không đủ: agent phù du, bán kính lớn, key chung không truy vết được.

- **SPIFFE** (CNCF): **X.509 SVID ngắn hạn, có attestation**.
- **Google Agent Identity** (4/2026): principal **hạng nhất**, tách khỏi human lẫn service account; cert xoay vòng, hạn **24h**.

Một agent phải xác thực về 4 hướng

- Tới **dịch vụ cloud**: bound token gắn cert
- Tới **tool/MCP ngoài**: API key / OAuth
- Tới tài nguyên **của user**: **3-legged OAuth**; tới **agent khác**: **mTLS + DPoP**

Lưu ý: OAuth 2.1 thuần **không diễn đạt được** "agent *nào* hành động thay user này". **Draft là draft** — nhớ Idempotency-Key: thiết kế để **thay được** cơ chế uỷ quyền.

Deployment Checklist

Một trang để soi trước khi bấm deploy — gói lại mọi thứ đã học hôm nay

Production Readiness Checklist

Container, Deploy & Agent

- ☐ Multi-stage + `uv`, <500MB, non-root
- ☐ `.dockerignore` + scan CVE (Trivy)
- ☐ Public URL + HTTPS hoạt động
- ☐ Env vars (không hardcode secret)
- ☐ Streaming (SSE) cho response dài
- ☐ State externalized (Redis/Postgres)

Security, Cost & Reliability

- ☐ Auth (API key / JWT)
- ☐ Rate limit + **spending cap** (admission control)
- ☐ Per-project/per-tenant budget
- ☐ `/health` (liveness + readiness)
- ☐ Graceful shutdown (drain)
- ☐ Rollback plan <2 phút
- ☐ **Pin model ID** + tắt auto-upgrade
- ☐ Router/fallback khi provider lỗi

Hoạt Động: Agent Của Bạn Deploy Ở Đâu? — 20 Phút

VINUNIVERSITY

Bước 1 — Chọn một use case

- Chatbot bán hàng — demo sau 1 tuần
- Agent nghiên cứu — 45 phút/tác vụ
- Trợ lý ngân hàng — data không rời VN
- Hoá đơn — 10 user, 2 lần/ngày
- App học — 100k user, dồn buổi tối
- **Hoặc: agent của chính nhóm bạn**

Bước 2 — Trình bày (tự do)

Chỉ **bắt buộc 2 điều**:

1. Deploy **ở đâu**?
2. **VÌ SAO** — ràng buộc nào quyết định?

Vẽ sơ đồ, slide, hay nói miệng — **không có mẫu**.

Lưu ý: “Chúng tôi chọn _____ **vì** _____.” Viết được câu này là xong.

Phụ Lục Thực Hành — Lệnh & Code

Gói lại thành thứ gõ được ngay: lệnh deploy thật, và một cost-guard tối thiểu — để rời lớp học là deploy được

Deploy Thật — Cloud Run & Railway

```
# A) Google Cloud Run: build from source, then deploy
gcloud run deploy agent-svc --source . \
  --port 8080 --concurrency 8 --memory 1Gi \
  --region asia-southeast1 --allow-unauthenticated \
  --set-env-vars MODEL_ID=claude-...,MAX_USD_PER_REQ=0.05 \
  --set-secrets ANTHROPIC_API_KEY=anthropic-key:latest

# B) Railway: deploy + set environment variables
railway up                          # streams build + deploy logs
railway variables --set "MODEL_ID=..." \
                  --set "ANTHROPIC_API_KEY=sk-..."
```

Lưu ý: `--concurrency` để **thấp** cho agent: mỗi request giữ một connection streaming dài. Secret tiêm qua `--set-secrets` / Variables tab — **không bao giờ** nhét vào image.

Cost-Guard Tối Thiểu — Chặn Trước Khi Gọi

```python
MAX_USD = float(os.environ["MAX_USD_PER_REQ"])  # e.g. 0.05

def guard(messages, model, user_id):
    in_tok = count_tokens(messages, model)      # count tokens
    est = in_tok/1e6*IN_PRICE + MAX_OUT/1e6*OUT_PRICE
    if est > MAX_USD:                           # admission control
        raise BudgetExceeded(f"${est:.3f} > ${MAX_USD}")
    metrics.tag(user=user_id, feature="chat", usd=est)
    return est
```

3 đòn bẩy FinOps

Tag mọi call, đếm token có thẩm quyền, **ép budget trước khi gọi LLM**. Nhớ: alert của provider là phản ứng *sau*; admission control chặn *trước*.

Hands-on & Key Takeaways

16

Mục tiêu cuối cùng rất cụ thể: agent có public URL, ai cũng truy cập được, có health check, có basic auth, có cost guard

Mục tiêu lab

Đóng gói agent thành container, deploy lên cloud, và có public URL hoạt động.

1. Viết Dockerfile (multi-stage + `uv`, slim base, non-root, $<$500MB)
2. Build & test container locally: `docker build` $\rightarrow$ `docker run` $\rightarrow$ scan bằng Trivy
3. Thêm streaming endpoint (SSE) + health check `GET /health`
4. Deploy lên Railway hoặc Render: connect repo $\rightarrow$ set env vars $\rightarrow$ deploy
5. Thêm basic auth (API key) + một rate limit / spending guard đơn giản
6. Demo: gửi request tới public URL, nhận response streaming từ agent

Blueprint Cần Nộp

VINUNIVERSITY

Container

- Dockerfile (multi-stage, uv, <500MB)
- docker-compose.yml + .dockerignore
- Health check + streaming endpoint
- Trivy scan sạch (no high CVE)

Deployment

- Public URL hoạt động (HTTPS)
- Env vars đúng cách (không hardcode)
- Basic auth (API key) + cost guard
- Demo request/response streaming

Lưu ý: Không cần enterprise-grade. Điều cần chứng minh là bạn **biết cách đưa agent từ localhost lên cloud**, nó hoạt động, và **không đốt tiền**.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **Cùng cỗ máy, khác cái hộp:** ship như thường — nhưng **the test** (eval gate), **the bill** (token), **the dependency** (model) đều bị định nghĩa lại.
2. **Container 2026:** multi-stage + `uv` + slim/distroless, non-root, scan CVE; SSE + externalize state.
3. **Chạy ở đâu = quyết định deploy:** server-side mặc định; "client-side" cần BFF proxy; keyless chỉ khi on-device.
4. **Platform theo timeout, scale theo concurrency;** router failover + cache cắt cost. Budget **KHÔNG** tự chặn → admission control + eval gate trong CI.
5. **Đừng mặc định:** framework và runtime là **hai tầng tách rời**; "model là API ngoài" là **lựa chọn**, không phải định luật.

Monitoring, Logging & Observability

“Agent deploy xong, 3 ngày sau: latency tăng gấp đôi, cost tăng 300%. Bạn không biết cho đến khi user phàn nàn. ”

- Đọc trước: LangSmith hoặc Langfuse quickstart (20 phút)
- Chuẩn bị: agent deployed từ Lab 12 cần có endpoint để gắn monitoring
- Suy nghĩ: metrics nào quan trọng nhất cho AI agent trên production?

1. Astral, *Using uv in Docker* — docs.astral.sh/uv/guides/integration/docker/. Multi-stage build hiện đại cho Python.
2. Google Cloud, *Cloud Run request timeout & concurrency* — cloud.google.com/run/docs. 60 phút max, concurrency 80/1000.
3. Model Context Protocol, *Transports & Authorization (**2026-07-28**)* — modelcontextprotocol.io. Streamable HTTP **stateless** + OAuth 2.1 + security best practices.
4. OWASP, *Top 10 for LLM Applications **2026*** (3/8/2026; 75% vote + 25% từ 6.639 sự cố thật) & *Top 10 for Agentic Applications 2026* — genai.owasp.org. LLM10; ASI03/ASI04.
5. Adam Wiggins, *The Twelve-Factor App* — 12factor.net. Factor VI (stateless processes).

Hỏi & Đáp

Từ hôm nay, agent không còn chỉ chạy trên máy bạn. Nó đã là một service thật sự — có URL, có bảo vệ, và không đốt sạch ngân sách.